The Foundation
0%

Module 1: QUICK START (...
1.1 What You Need (5 Min Set...
1.2 Your First Folder
1.3 How to Structure Any Pro...

Module 2: WHY THIS WO...
2.1 Video Text Guide (Series ...
2.2 One Line of Python Trigge...
2.3 How a 1953 Word Game E...
2.4 The Ladder That Explains ...
2.5 Clawdbot (Moltbot) Has 1...
2.6 Video as Code: My AI Ani...
2.7 From Nazi Psychology to ...

Module 3: FOLDER ARCHI...
3.1 The Full Walkthrough (23 ...
3.2 Customizing for Your Use ...
3.3 Common Mistakes and H...

3.3 Common Mistakes and How to Fix Them

What You'll Get From This

The seven mistakes people make most often when setting up the folder architecture, and how to fix each one before it wastes your time.

The Lesson

Every one of these comes from real community members. People post their setups, run into walls, and ask for help. The same problems show up over and over. If you read this before you finalize your structure from 3.2, you will skip most of them.

Mistake 1: Making the CLAUDE.md too long

The CLAUDE.md is a routing file. It tells Claude where things are and where to go. That is its job. It is not a project brief. It is not a style guide. It is not a brain dump of everything you want Claude to know.

When the CLAUDE.md gets too long, two things happen. First, Claude burns tokens reading information that is not relevant to the current task. Second, the signal gets buried in noise. The routing instructions (the part that actually matters) get lost in paragraphs of background context.

The fix: Your CLAUDE.md should fit on one screen. Identity, folder structure, routing table, naming conventions. That is it. Everything else belongs in a workspace CONTEXT.md where it only gets loaded when Claude is actually working in that area. If

Mistake 2: Skipping the routing table

Some people set up the folders, write the context files, and never put a routing table in the CLAUDE.md. They assume Claude will figure it out.

Sometimes Claude does figure it out. But "sometimes" is the problem. Without a routing table, Claude has to guess which files to read and which to skip. It might read everything (wasting tokens) or read the wrong context file (getting confused about what workspace it is in). The output gets inconsistent. Some responses are great, some are off, and you cannot tell why.

The fix: The routing table does not need to be complicated. Three columns: task, where to go, what to read. One row per type of work. Claude reads the table, matches the task, goes to the right place. Every time.

```
| Task | Go to | Read |
|------|-------|------|
| Write content | /script-lab | CONTEXT.md |
| Build something | /production | CONTEXT.md |
| Publish or schedule | /distribution | CONTEXT.md |
```

If you added skills (Layer 3), add a fourth column. Otherwise, three is enough.

Mistake 3: Too many workspaces

Someone sets up eight workspaces for a project that really only has two or three modes of work. They have a workspace for research, a workspace for notes, a workspace for drafts, a workspace for editing, a workspace for final review, a workspace for publishing. Each one has its own CONTEXT.md. The CLAUDE.md has a routing table with twelve rows.

Now the overhead of maintaining the system is bigger than the work itself. Context files go stale. Claude spends more time

The fix: Start with two or three workspaces. You can always add more. The question to ask: "Do I shift mental modes between these tasks?" Writing and building are different mental modes. That is two workspaces. Drafting and editing are the same mental mode at different stages. That is one workspace with a process inside it, not two workspaces.

If you are not sure whether something deserves its own workspace, it does not. Keep it as a subfolder inside an existing workspace. You can always split it out later if the work grows.

Mistake 4: Writing context files that describe what AI should be instead of what the work is

This one is subtle. People write context files full of instructions about Claude's personality. "Be creative. Be concise. Be professional. Think step by step. Use a warm tone." They spend 30 lines describing how Claude should behave and two lines describing the actual project.

Claude responds to context about the work far more than context about itself. Telling Claude "you are a senior copywriter" gives it a role. Telling Claude "the audience is mid-market HR directors who have tried three other tools and are skeptical of AI claims" gives it something to actually work with. The second one changes the output more than the first.

The fix: Flip the ratio. Spend 80% of your context file describing the work: what the project is, who the audience is, what has already been done, what good output looks like, what to avoid. Spend 20% or less on behavioral instructions. If your context file reads like a personality quiz, rewrite it. If it reads like a project brief that a new team member could pick up and start working from, you are in the right place.

Mistake 5: Never updating the context files

Someone sets up the folder in week one. Writes the context files. Gets great results for two weeks. Then the project evolves.

member thinks Claude "got worse" when really Claude is doing exactly what the context tells it to do. The context is just stale.

The fix: Treat context files like working notes. When the project changes, edit the context. When you learn something new about what Claude needs to know, add it. When a constraint no longer applies, remove it. This takes 30 seconds per edit. It is the single highest-leverage habit in the whole system.

Some people add a "Last updated" line at the top of each context file. Simple but effective. When you open a workspace and see "Last updated: six weeks ago," you know to review it before working.

Mistake 6: Putting everything in one folder and hoping the CLAUDE.md sorts it out

The opposite of too many workspaces. Everything lives in one flat directory. Fifty files. No subfolders. The CLAUDE.md tries to route between them using file names alone. Claude reads the whole directory listing, picks what it thinks is relevant, and often picks wrong.

This is the equivalent of dumping every document on one desk and asking someone to find the right one. They can do it. It just takes longer, they make more mistakes, and they get frustrated.

The fix: If you have more than 8-10 files at the same level, you need subfolders. Group by workspace (what kind of work), then by stage or type within the workspace. The folder structure is the architecture. It tells Claude what belongs together. Let it do that job.

Mistake 7: Building the whole system before using it

The most common mistake. Someone reads 3.1 and 3.2, gets inspired, and spends an entire weekend building a perfect folder architecture with six workspaces, detailed context files, naming conventions, skills wired into every workspace, and a routing

Then they start using it and realize half the decisions they made do not match how they actually work. The workspace boundaries are wrong. The context files describe what they thought they would need, not what they actually need. They have to rebuild.

The fix: Build the minimum. One CLAUDE.md, one or two workspaces, one CONTEXT.md per workspace. Start working. After a few days, you will know what is missing. Add it then. After a week, you will know what is wrong. Fix it then.

The best folder setups in the community were all built incrementally. They started simple and grew based on real work. The ones that stalled were the ones that tried to be complete before day one.

Your first version should take 15 minutes. If it took longer, you over-built.

The pattern across all seven

If you look at these together, they all point in the same direction. Keep the system small. Keep it focused on the work, not on Claude. Update it as you go. Let the structure grow from use, not from planning.

The folder architecture is powerful because it is simple. Folders and text files. The moment it starts feeling heavy or complicated, something went wrong. Go back to the three layers from 3.1: map, rooms, tools. If your CLAUDE.md is the map, your context files are the rooms, and your skills are the tools, and each one is doing only its job, the system stays clean.

Go Deeper

- If you have not built your structure yet, go back to **3.2: Customizing for Your Use Case** and start from one of the three examples. Build the minimum version. Come back here when you run into something.

task stuff. Separating those two keeps your context files focused on the work.

- The production pipeline in **2.6: Video as Code** is an example of a system that grew incrementally. It started as a simple folder and became a full spec-to-animation pipeline over weeks of use. That is the right pace.
- **The Vault** (Premium) has the full Blueprints with example context files already written. If you want a starting point that is further along than the quick-start three files but still customizable, that is what it is for.

Next Up: Section 4 — Claude Code (Getting Hands On)

You have the architecture. You know how to customize it and what to avoid. Section 4 gets you into Claude Code, the terminal tool that makes all of this run. If you have been using Claude Projects or pasting context into the browser, Section 4 is where you level up.

Matthew Creamer
24d • Lesson Check-Ins

Foundations 3.3 Check-In

Everyone gets something wrong the first time. Vote below, then drop your mistake in the comments so others can

Poll 94 members have voted

34 57 New comment 2h ago